

Chapter 10

Numerical Differentiation and Integration, Spline Fits

We have, over the past weeks, spent considerable effort fitting functions to data, to uncover or verify relationships. The methods of curve fitting work well (even if finding the best fit might require quite a bit of trial and error to find good initial guesses) in cases where the underlying relationship can be expressed algebraically, or with a reasonable Taylor-series approximation.

There are types of solutions, however, that cannot be expressed algebraically. For example, we could measure the force applied to an object, and the resulting kinetic energy change. This is an application of the work-kinetic-energy theorem,

$$\int \vec{F} \cdot d\vec{s} = \Delta K = \frac{1}{2}m(v^2 - v_0^2)$$

where the *integral* of our measured quantity (the force) is a quadratic function of the speed. Similarly, we can measure the position of an object as a function of time (we have done this in several different cases). Rather than fit this position data, we may be interested in the *velocity* of the object (perhaps as part of a kinetic energy measurement, as above) or the *acceleration* of the object (since measuring force and acceleration allows us to determine mass).

$$\vec{v} = \frac{d\vec{x}}{dt}, \quad \vec{a} = \frac{d^2\vec{x}}{dt^2}$$

Here, the *derivative* of our measured quantity is of interest.

There are other cases where our only interest is *interpolation* in between data points. Of course, we could attempt to find an overall function fit, and evaluate this fit at our desired interpolation point. In many cases, however, we may not be able to find an overall function that describes the entire range of interest (because there can be present more than one effect, and so we may have to define piecewise fits, for example). If our data has minimal uncertainty, then we can construct a piecewise fit called a *spline fit* that allows interpolation to be readily performed.

Numerical Integration of Data

To numerically integrate data, we recall the definition of an integral as a Reimann sum.

$$\int f(x) dx = \lim_{\delta x \rightarrow 0} \sum f(x_i) \Delta x$$

Since measured data is always discrete, the limit in this context does not make sense. Omitting it, we have an approximation

$$\int f(x) dx \approx \sum f(x_i) \Delta x = \sum f(x_i)(x_{i+1} - x_i)$$

Since the integral is an area under a curve, we can approximate this area as the sum of areas of rectangles of height $f(x_i)$ and width $\Delta x = x_{i+1} - x_i$. We recognize here that this is just Euler's method, that we covered in Physics 150, and reviewed back in Week 2. This will give a good estimate for the integral if the $\Delta x = x_{i+1} - x_i$ is small enough. Unfortunately, using measured discrete data points, we cannot always make Δx small enough. The minimum value we can have for Δx is the difference between consecutive values of the data, and so this simple method can result in a noticeable error (especially for the case of monotonically increasing or decreasing functions).

An alternative is to use a trapezoidal approximation – that is, we can approximate the area under the data curve as the sum of areas of *trapezoids*. Trapezoids have the distinct advantage that they approximate the area by evaluating at *both* ends of the interval, instead of just at the beginning. Recall the area of a trapezoid:

$$A = \frac{1}{2}(b_1 + b_2)h$$

For our discrete data, this takes the form

$$\frac{1}{2}(y_{i+1} + y_i)\Delta x = \frac{1}{2}(y_{i+1} + y_i)(x_{i+1} - x_i)$$

and so our integral is approximated as

$$\int y dx = \frac{1}{2} \sum [(y_{i+1} + y_i)(x_{i+1} - x_i)]$$

This results in an integral with, typically, substantially less accumulated error, especially for monotonically increasing or decreasing data sets¹.

Another method that can be used is to first fit the data to a function, and then integrate that fit function. This method works well, provided we trust our fit function. This potentially can introduce bias into our result if the fit function is not really the appropriate function. Using an appropriate fit function does have the distinct advantage that the least-squares curve fitting process is statistically more robust, since it tends to average out random errors from the data set in ways that direct integration of the data points does not always accomplish well.

Numerical Differentiation of Data

To numerically differentiate data, we recall the definition of a derivative:

$$y' = \frac{dy}{dx} = \lim_{\Delta x \rightarrow 0} \frac{\Delta y}{\Delta x}$$

¹Higher-order methods, such as Simpson's rule or Runge-Kutta methods can also be employed. While these typically offer better results, they are substantially more complicated to implement. We will leave these for Physics 385.

For discrete data, we omit the limit to get an approximation

$$y'_i \approx \frac{\Delta y}{\Delta x} = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$$

Since we are using each end of an interval to calculate the change, we note two important consequences: we have one fewer values for the derivative than we had original data points; and, this numerical differentiation does not calculate the derivative at either end. Instead, this is best approximated as the derivative at the *midpoint* of the interval,

$$x_{mid} = \frac{1}{2}(x_{i+1} + x_i)$$

This can be inconvenient, since the midpoint may not correspond to a measured data point. To get around this difficulty, we can do one of two things. First, we can define a new independent data set,

$$\left\{ \frac{x_1 - x_0}{2}, \frac{x_2 - x_1}{2}, \dots \right\}$$

This set will have one fewer point than the original set of x values, just as the set of derivatives will have one fewer point than the original set of y values. Second, we can use an interval of *three* data points,

$$y'_i = \frac{y_{i+1} - y_{i-1}}{x_{i+1} - x_{i-1}} \quad (1)$$

and this corresponds to the point x_i . This results in a set of derivatives that has *two* fewer points (there is no derivative that corresponds to x_0 or x_N), and so we associate this set of derivatives with the abbreviated list

$$x_{abr} = \{x_1, x_2, \dots, x_{N-1}\}$$

Which of these methods we pick is largely a matter of convenience in how we represent our results.

In each of these cases, we note that calculating a derivative involves computing a *difference* between two measured values that usually are not very different from each other. For example, we could have point $y_n = 2.03 \pm 0.02$ and $y_{n+1} = 2.14 \pm 0.03$. Each of these points has three significant figures, and the fractional uncertainties in these are

$$\frac{\delta y_n}{y_n} = \frac{0.02}{2.03} = 0.0099, \quad \frac{\delta y_{n+1}}{y_{n+1}} = \frac{0.03}{2.14} = 0.014$$

If we now compute the difference,

$$\Delta y_n = y_{n+1} - y_n = 2.14 - 2.03 = 0.11$$

and compute its uncertainty

$$\delta(\Delta y_n) = \sqrt{(\delta y_{n+1})^2 + (\delta y_n)^2} = \sqrt{(0.02)^2 + (0.03)^2} = 0.04$$

The difference is

$$\Delta y_n = 0.11 \pm 0.04$$

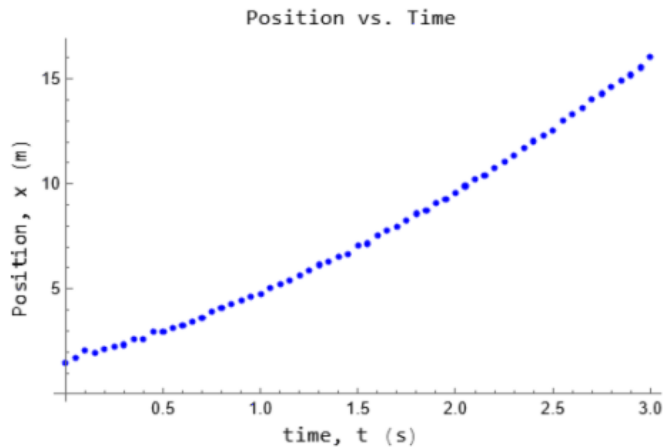
We note that the difference has only two significant figures. The fractional uncertainty is

$$\frac{\delta \Delta y_n}{\Delta y_n} = \frac{0.04}{0.11} = 0.33$$

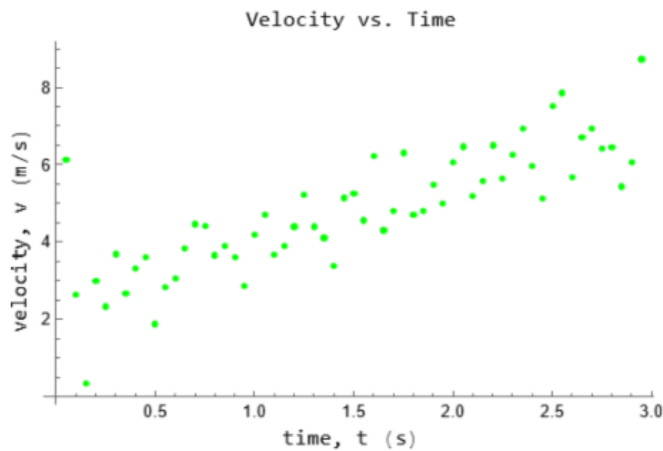
which is substantially larger than it was for either of the data points. This illustrates an important complication with numerically differentiating data:

The numerical derivative of a set of data typically has quite a bit larger uncertainty associated with it than does the original data.

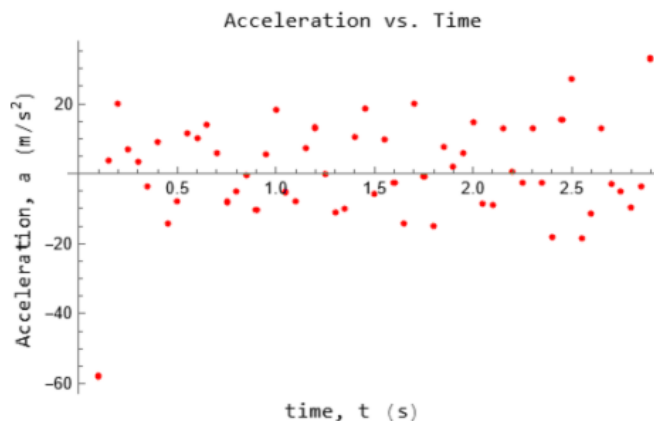
To illustrate, suppose some position-vs-time data is collected, and plotted



This shows a fairly clear quadratic trend, suggesting a constant acceleration. Some experimental uncertainty is evident. If the raw data is numerically differentiated, the velocity-vs-time plot that results,



shows noticeably more scatter, indicating a greater uncertainty. If we then numerically differentiate this again to plot acceleration vs time,



there is now so much scatter in the twice-differentiated data that we cannot even be confident that the result is positive.

As with integration of data, we can use a fit function to avoid this complication. If we first fit our raw data to a function, and then differentiate this function, we get much more satisfactory results. The reason for this is that the fit function has used *all* of the data points to find a best fit, and so has resulted in a lot more cancellation of random errors. The numerical differentiation, on the other hand, is only using two data points – meaning that very little cancellation of random errors is occurring

Smoothing Data

We can, in some cases, average out random errors by employing *data smoothing*. The idea here is to average several adjacent points together in an attempt to force some cancellation of random errors. This, of course, would be rather problematic if the data is rapidly changing from point to point. So, we will not expect that any data smoothing should be done unless we either have data that is only slowly changing, or data that has been very finely sampled (so that differences between adjacent data points are not very large compared with the value of either point). This works especially well for data sets that are approximately constant or approximately linear. The idea is that averaging over a few consecutive data points will tend to result in more cancellation of uncertainty than addition of uncertainty, relying on simple probability that random errors will likely not be in the same direction three or four or five times in a row. So, instead of our original data, we can report a *moving* average of several consecutive data points. In doing this, the moving average corresponds to the midpoint of the smoothing range (similar to how a numerical derivative is associated with the midpoint of the interval). As a result, it is typical to choose to smooth over an *odd* number of points, so that the midpoint will correspond to a data point. For example, suppose we smooth over three data points. We then have

$$y_1^{(s)} = \frac{1}{3}(y_0 + y_1 + y_2)$$

$$y_2^{(s)} = \frac{1}{3}(y_1 + y_2 + y_3)$$

and so forth. These smoothed data points would then be plotted vs. their midpoint (so the smoothed data point $y_1^{(s)}$ is plotted vs. x_1 , $y_2^{(s)}$ is plotted vs. x_2 , and so forth).

If errors are truly random, then averaging over more points will tend to average them closer to zero. So, it would seem that a wider smoothing range would reduce noise more effectively. However, a moment's thought makes it quite apparent that this is really only true if the data is a repeated measurement of a *constant* quantity. If the data has a trend in it, *any* smoothing is, strictly speaking, wrong. But, if the random scatter in the data is larger than the trend from one point to the next, then smoothing the data can be helpful. Broader smoothing ranges also increasingly run the risk of smoothing out features in the data – especially features that are of very short duration. Trends in smoothed data are also notorious for being somewhat sensitive to the smoothing range (either how wide the range is, or where the beginning point of the range is chosen). In all, data smoothing should be used with caution.

Splines

Splines are an alternative to curve fitting in cases where our only interest is *interpolation* in between data points (that is, splines do *not* work for extrapolation beyond the ends of a data set). We note (and this is an important point) that splines also are not appropriate for data that has noise in it.

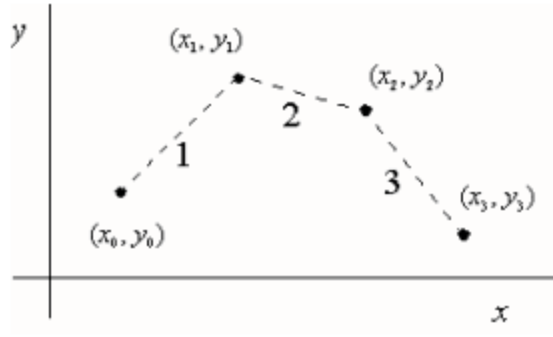
In order to use splines, *we must begin with precise data.*

If the data still has noticeable noise in it, increased experiment time, or some data smoothing might be appropriate before using splines.

At their most fundamental, splines are a method of connecting the dots. There is no minimization of χ^2 , so there is no reduction of uncertainty. Indeed, if noticeable uncertainty is still present, splines are likely to result in wildly fluctuating interpolations, since splines only ever connect the dots.

The simplest spline fit is a *linear* spline. This is exactly the connect-the-dots exercise you learned back perhaps even pre-Kindergarten: we simply fit a straight line between each pair of consecutive data points. This means that if we have 10 data points then we will have nine splines. That is, we will have nine straight-line equations, one for each interval $\{x_i, x_{i+1}\}$. Interpolation between data points is then done by selecting the appropriate line equation from the set of all the linear spline equations, and evaluating it at the desired point.

To illustrate, consider a simple case of four data points, plotted as shown.



For these four data points, we connect consecutive pairs, and so we need *three* splines. For this linear case, we can use the two-point form of a line,

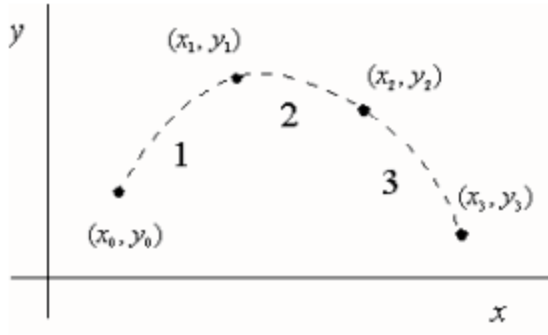
$$y - y_n = \frac{y_{n+1} - y_n}{x_{n+1} - x_n} (x - x_n)$$

applied three times:

$$\begin{aligned} \underline{x_0 < x < x_1} : \\ y - y_0 &= \frac{y_1 - y_0}{x_1 - x_0} (x - x_0) \\ \underline{x_1 < x < x_2} : \\ y - y_1 &= \frac{y_2 - y_1}{x_2 - x_1} (x - x_1) \\ \underline{x_2 < x < x_3} : \\ y - y_2 &= \frac{y_3 - y_2}{x_3 - x_2} (x - x_2) \end{aligned}$$

Again we emphasize that splines are not used for extrapolation, and so there are no equations defined for $x < x_0$ or $x > x_3$. For this case of *linear* splines, we note that we can directly write down the final equations rather easily, and so a linear spline is simple to implement. This simplicity comes at a noticeable cost: the slope abruptly changes at each point (known as a *knot*). If the trend in the data is close to linear, this will not be a large problem. In the case illustrated here, that is not close to linear, assuming a constant slope in between knots could easily lead to significant interpolation error. So, overall we see that linear splines are easy to implement but, because they have disjointed slopes, do not offer very good interpolation except for the case of data that is constant or linear or that has been finely sampled (closely-spaced data points).

The next option is is a *quadratic* spline. This method fits *parabolas* in between each pair of consecutive data points.



Each spline is a second-order polynomial of the form

$$y = a_n x^2 + b_n x + c_n$$

And so each spline has three parameters to find. With n data points, we have a total of $3n$ parameters, and so we need $3n$ equations relating these parameters to each other.

First, we require that the splines pass through the data points – that is, each spline meets the knots at each end. Each spline is evaluated at two points, resulting in $2n$ equations. For our example of four data points, the first spline is evaluated at (x_0, y_0) and at (x_1, y_1) , the second spline is evaluated at (x_1, y_1) and (x_2, y_2) , and the third spline is evaluated at (x_2, y_2) and (x_3, y_3) :

$$\left. \begin{aligned} y_0 &= a_1 x_0^2 + b_1 x_0 + c_1 \\ y_1 &= a_1 x_1^2 + b_1 x_1 + c_1 \end{aligned} \right\} \text{ first spline}$$

$$\left. \begin{aligned} y_1 &= a_2 x_1^2 + b_2 x_1 + c_2 \\ y_2 &= a_2 x_2^2 + b_2 x_2 + c_2 \end{aligned} \right\} \text{ second spline}$$

$$\left. \begin{aligned} y_2 &= a_3 x_2^2 + b_3 x_2 + c_3 \\ y_3 &= a_3 x_3^2 + b_3 x_3 + c_3 \end{aligned} \right\} \text{ third spline}$$

The second and third of these equations ensure the continuity at x_1 (that is, the first spline at its right end is equal to the second spline at its left end), and the fourth and fifth equations ensure the continuity at x_2 . Overall, we obtain 6 equations of the 9 required.

The quadratic form allows the interpolating functions to have a continuous slope across each data point. That is, at each knot we require the first derivatives to be the same,

$$2a_1 x_1 + b_1 = 2a_2 x_1 + b_2$$

$$2a_2 x_2 + b_2 = 2a_3 x_2 + b_3$$

This matching of slopes cannot be done for the first or the last knots, and results in $n - 1$ equations (2, for our example here). Thus, we have found $3n - 1$ equations, and so need one more.

There is not a unique way to obtain the last equation, but there are some options. First, we can assume that the curvature at the first knot at (x_0, y_0) is zero, so that

$$a_1 = 0$$

This has the effect of reducing the first spline to linear. While this is a common choice, it is somewhat arbitrary. We could just as well have chosen the *last* knot (x_n, y_n) to have zero curvature, and so have

$$a_n = 0$$

Another option is to pick one of the interior knots and force the curvature to be the same so that

$$a_n = a_{n+1}$$

Which should we pick? If the trend in the data is approximately linear over the first few points, choosing the first spline to be linear is probably a good choice. If the final few points are approximately linear, then choosing the last spline to be linear makes sense. If neither appears very linear, then we could pick an interior knot to have continuity of curvature. If these different choices do not result in noticeable differences in an interpolated value, then the choice of our last equation is not very important (this is fairly commonly the case). If, on the other hand, noticeable differences *do* occur, we probably conclude that the data is not as well sampled as we hope, and we need a few more points in our data set. If collecting new data points is not feasible, we could try several different interpolations, and average their results.

The continuity of slope (in addition to continuity) offers better interpolation performance. There is still a discontinuity of the curvature (the second derivative) at each data point, and so any calculation that relies on evaluating derivatives may have issues with quadratic splines.

In practice, the most commonly used spline fit is *cubic*. As the name suggests, this involves fitting a cubic polynomial ($a_n x^3 + b_n x^2 + c_n x + d_n$) to each set of consecutive data points (that is, there are *four* parameters per spline, so that we need $4n$ equations). This allows both slope and curvature to be continuous, and so offers even better interpolation performance.

Each spline is evaluated at the knot at each end of the spline, resulting in $2n$ equations. The slope (first derivative) and curvature (second derivative) of consecutive splines are matched at each interior knot, resulting in $2(n - 1) = 2n - 2$ additional equations. This leaves two additional equations needed. Typically, the last two equations are obtained by setting the curvature (that is, the second derivative) to zero at both end points,

$$\begin{aligned} S_1''(x_1) &= 6a_1x_1 + 2b_1 = 0 \\ S_n''(x_n) &= 6a_nx_n + 2b_n = 0 \end{aligned}$$

For our example above, we need 4 parameters for each spline, and so need 12 equa-

tions. Evaluating at each end point of the spline results in 6 equations,

$$\begin{aligned}
y_0 &= a_1x_0^3 + b_1x_0^2 + c_1x_0 + d_1 \\
\left. \begin{aligned} y_1 &= a_1x_1^3 + b_1x_1^2 + c_1x_1 + d_1 \\ y_1 &= a_2x_1^3 + b_2x_1^2 + c_2x_1 + d_2 \end{aligned} \right\} && \text{match first spline to the second} \\
\left. \begin{aligned} y_2 &= a_2x_2^3 + b_2x_2^2 + c_2x_2 + d_2 \\ y_2 &= a_3x_2^3 + b_3x_2^2 + c_3x_2 + d_3 \end{aligned} \right\} && \text{match second spline to the third} \\
y_3 &= a_3x_3^3 + b_3x_3^2 + c_3x_3 + d_3
\end{aligned}$$

Matching the slope (the first derivative) at each knot results in two more equations,

$$\begin{aligned}
3a_1x_1^2 + 2b_1x_1 + c_1 &= 3a_2x_1^2 + 2b_2x_1 + c_2 \\
3a_2x_2^2 + 2b_2x_2 + c_2 &= 3a_3x_2^2 + 2b_3x_2 + c_3
\end{aligned}$$

Matching the curvature (the second derivative) at each knot results in two more equations,

$$\begin{aligned}
6a_1x_1 + 2b_1 &= 6a_2x_1 + 2b_2 \\
6a_2x_2 + 2b_2 &= 6a_3x_2 + 2b_3
\end{aligned}$$

Finally, setting the curvature to zero at each end results in

$$\begin{aligned}
6a_1x_0 + 2b_1 &= 0 \\
6a_3x_3 + 2b_3 &= 0
\end{aligned}$$

Solving for the parameters is a straightforward (if tedious) process.

For our purposes, we will not attempt to calculate any of these spline fits by hand, but instead will note that Mathematica and Python both have built-in spline routines.